

cost_change_functions_documentation

Fred Cudhea

26 March, 2025

R Markdown

Documentation for R script: cost_change_functions.r which contains various functions used in calculating and summarizing the impacts for the non-health pillar outcomes.

The first function here, simulate.impact, is the core of the entire operation. It runs monte carlo simulations for env/cost/labor impact calculations and outputs the resulting distributions.

Meaning of 21 inputs should be mostly self explanatory, but 2 things may need explanation. 1.

“substitution.impact.factor.means”, “substitution.impact.factor.ses”, “substitution_unit” refer to substitution impact.

While we don’t use this right now (all these inputs are zero), we can calculate substitution effects as well. Say, if we change the diet for just one diet factor, we need replace the calories from other foods. So, we need to calculate the impact of the increase/decrease that comes from the substitution (probably using the impact per unit of the average diet as substitution impact factor). Since we keep energy constant for the five alternate scenarios, we don’t actually use this for the 4 pillars paper. But if you’re not keeping energy constant when going from current to counterfactual intake, you’d want to use substitution effects. 2. “current_foodwaste_p”, “current_foodwaste_p_se”, “counterfactual_foodwaste_p”, “counterfactual_foodwaste_p_se”: Don’t forget that food waste proportion is conditional on edible. It’s the proportion of the food wasted out of the edible portion. It is NOT food waste / total food including inedible portion.

```
simulate.impact<-function(current.mean, current.se,
                           counterfactual.mean, counterfactual.se=0,
                           impact.factor.names, impact.factor.means,
                           impact.factor.ses, RRunit,
                           substitution.impact.factor.means,
                           substitution.impact.factor.ses, substitution_unit,
                           current_inedible_p, current_inedible_p_se,
                           current_foodwaste_p, current_foodwaste_p_se,
                           counterfactual_inedible_p, counterfactual_inedible_p_se,
                           counterfactual_foodwaste_p, counterfactual_foodwaste_p_se,
                           n.sims,
                           population.sims ##<- must be a vector, so make sure to convert to vector
)
{
```

First simulate intake means for current and counterfactual scenarios. We are creating a matrix of sims because we are going to use them to do calculations for all outcomes from three pillars (with each row corresponding to an outcome of interest).

```
# current.mean.sims<-rnorm(n=n.sims,
#                           mean=current.mean,
#                           sd=current.se)
# counterfactual.mean.sims<-rnorm(n=n.sims,
#                                  mean=counterfactual.mean,
#                                  sd=counterfactual.se)

current.mean.sims<-rnorm(n=n.sims,
                        mean=current.mean,
                        sd=current.se)
current.mean.sims.matrix<-t(matrix(rep(current.mean.sims, times=length(impact.factor.names)),
                                   ncol=length(impact.factor.names)))
counterfactual.mean.sims<-rnorm(n=n.sims,
                               mean=counterfactual.mean,
                               sd=counterfactual.se)
counterfactual.mean.sims.matrix<-t(matrix(rep(counterfactual.mean.sims, times=length(impact.factor.names)),
                                           ncol=length(impact.factor.names)))
```

Calculate **correction factors** (from inedible and waste proportion) to go from current intake to edible amount and total produced (edible + inedible).

```
#current.correction<-1/((1-rnorm(n.sims, mean=current_inedible_p, sd=current_inedible_p_se))*(1-
#counterfactual.correction<-1/((1-rnorm(n.sims, mean=counterfactual_inedible_p, sd=counterfactual_inedible_p_se))

current.correction<-1/((1-rnorm(length(impact.factor.names), mean=current_inedible_p, sd=current_inedible_p_se))
counterfactual.correction<-1/((1-rnorm(length(impact.factor.names), mean=counterfactual_inedible_p, sd=counterfactual_inedible_p_se))

# UNCOMMENT LATER
# ##this is a correction factor to get total edible. "current.correction" above is for getting t
current.correction.for.waste<-1/(1-rnorm(length(impact.factor.names), mean=current_foodwaste_p, sd=current_foodwaste_p_se))
counterfactual.correction.for.waste<-1/(1-rnorm(length(impact.factor.names), mean=counterfactual_foodwaste_p, sd=counterfactual_foodwaste_p_se))
```

Apply corrections, and based on that, calculate unconsumed, edible, inedible and wasted amounts.

```
#current.mean.sims.after.correction<-current.mean.sims*current.correction
#counterfactual.mean.sims.after.correction<-counterfactual.mean.sims*counterfactual.correction

current.mean.sims.matrix.after.correction<-current.mean.sims.matrix*current.correction
counterfactual.mean.sims.matrix.after.correction<-counterfactual.mean.sims.matrix*counterfactual

current.mean.sims.matrix.unconsumed<-current.mean.sims.matrix.after.correction-current.mean.sims
counterfactual.mean.sims.matrix.unconsumed<-counterfactual.mean.sims.matrix.after.correction-cou

## UNCOMMENT LATER
current.mean.sims.matrix.edible<-current.mean.sims.matrix*current.correction.for.waste
counterfactual.mean.sims.matrix.edible<-counterfactual.mean.sims.matrix*counterfactual.correcti

current.mean.sims.matrix.inedible <- current.mean.sims.matrix.after.correction - current.mean.s
counterfactual.mean.sims.matrix.inedible <- counterfactual.mean.sims.matrix.after.correction -

current.mean.sims.matrix.wasted <- current.mean.sims.matrix.edible - current.mean.sims.matrix
counterfactual.mean.sims.matrix.wasted <- counterfactual.mean.sims.matrix.edible - counterfactu
```

Calculate deltas (difference between current and counterfactual)

```
#delta.sims<-counterfactual.mean.sims-current.mean.sims
#delta.sims<-counterfactual.mean.sims.after.correction-current.mean.sims.after.correction

delta.sims.matrix<-counterfactual.mean.sims.matrix.after.correction-current.mean.sims.matrix.aft

delta.sims.matrix.consumed<-counterfactual.mean.sims.matrix-current.mean.sims.matrix
delta.sims.matrix.unconsumed<-counterfactual.mean.sims.matrix.unconsumed-current.mean.sims.matri
delta.sims.matrix.total<-delta.sims.matrix.consumed+delta.sims.matrix.unconsumed

# UNCOMMENT LATER
delta.sims.matrix.inedible<-counterfactual.mean.sims.matrix.inedible-current.mean.sims.matrix.i
delta.sims.matrix.wasted<-counterfactual.mean.sims.matrix.wasted-current.mean.sims.matrix.waste
```

Simulate impact factors.

```
#population.sims<-rnorm(n=n.sims, mean=population, sd=population.se)

impact.factor.sims<-matrix(rnorm(n=n.sims*length(impact.factor.names),
                                mean=as.numeric(impact.factor.means),
                                sd=as.numeric(impact.factor.ses)),
                           nrow=length(impact.factor.names))

substitution.impact.factor.sims<-matrix(rnorm(n=n.sims*length(impact.factor.names),
                                                mean=as.numeric(substitution.impact.factor.means),
                                                sd=as.numeric(substitution.impact.factor.ses)),
                                         nrow=length(impact.factor.names))
```

Calculate impact and assign row names to the output matrices (for consumed, unconsumed, total, inedible, and wasted)

```
#impact.sims <- t(t(impact.factor.sims)*delta.sims*t(population.sims))/as.numeric(RRunit)

#impact.sims <- impact.factor.sims*delta.sims.matrix*population.sims/as.numeric(RRunit)

impact.sims.consumed <- impact.factor.sims*delta.sims.matrix.consumed*population.sims/as.numeric(RRunit)
impact.sims.unconsumed <- impact.factor.sims*delta.sims.matrix.unconsumed*population.sims/as.numeric(RRunit)
impact.sims.total<-impact.sims.consumed+impact.sims.unconsumed

# UNCOMMENT LATER
impact.sims.inedible <- impact.factor.sims*delta.sims.matrix.inedible*population.sims/as.numeric(RRunit)
impact.sims.wasted <- impact.factor.sims*delta.sims.matrix.wasted*population.sims/as.numeric(RRunit)

#rownames(impact.sims)<-impact.factor.names
rownames(impact.sims.consumed)<-impact.factor.names
rownames(impact.sims.unconsumed)<-impact.factor.names
rownames(impact.sims.total)<-impact.factor.names

# UNCOMMENT LATER
rownames(impact.sims.inedible)<-impact.factor.names
rownames(impact.sims.wasted)<-impact.factor.names
```

Calculate substitution impact and assign row names to the output matrices

```
#substitution.sims<-t(t(substitution.impact.factor.sims)*(-delta.sims)*t(population.sims))/substitution.impact.factor.names

#substitution.sims<-substitution.impact.factor.sims*-delta.sims.matrix*population.sims/substitution.impact.factor.names

substitution.sims.consumed<-substitution.impact.factor.sims*-delta.sims.matrix.consumed*population.sims/substitution.impact.factor.names
substitution.sims.unconsumed<-substitution.impact.factor.sims*-delta.sims.matrix.unconsumed*population.sims/substitution.impact.factor.names
substitution.sims.total<-substitution.sims.consumed+substitution.sims.unconsumed

# UNCOMMENT LATER
substitution.sims.inedible<-substitution.impact.factor.sims*-delta.sims.matrix.inedible*population.sims/substitution.impact.factor.names
substitution.sims.wasted<-substitution.impact.factor.sims*-delta.sims.matrix.wasted*population.sims/substitution.impact.factor.names

#rownames(substitution.sims)<-impact.factor.names
rownames(substitution.sims.consumed)<-impact.factor.names
rownames(substitution.sims.unconsumed)<-impact.factor.names
rownames(substitution.sims.total)<-impact.factor.names

# UNCOMMENT LATER
rownames(substitution.sims.inedible)<-impact.factor.names
rownames(substitution.sims.wasted)<-impact.factor.names
```

Combine impact and substitution impact to get total impact

```
#combined.impact.sims<-impact.sims+substitution.sims

combined.impact.sims.consumed<-impact.sims.consumed+substitution.sims.consumed
combined.impact.sims.unconsumed<-impact.sims.unconsumed+substitution.sims.unconsumed
combined.impact.sims.total<-combined.impact.sims.consumed+combined.impact.sims.unconsumed

# UNCOMMENT LATER
combined.impact.sims.inedible<-impact.sims.consumed+substitution.sims.inedible
combined.impact.sims.wasted<-impact.sims.unconsumed+substitution.sims.wasted
```

Calculate impact of current and counterfactual intake on the outcomes (as opposed to the the impact of the shift from current to counterfactual)

```

#combined.impact.sims<-impact.sims+substitution.sims

combined.impact.sims.consumed<-impact.sims.consumed+substitution.sims.consumed
combined.impact.sims.unconsumed<-impact.sims.unconsumed+substitution.sims.unconsumed
combined.impact.sims.total<-combined.impact.sims.consumed+combined.impact.sims.unconsumed

# UNCOMMENT LATER
combined.impact.sims.inedible<-impact.sims.consumed+substitution.sims.inedible
combined.impact.sims.wasted<-impact.sims.unconsumed+substitution.sims.wasted

###total impact/cost, not just difference between current and counterfactual
current.impact.sims<-impact.factor.sims*current.mean.sims.matrix.after.correction*population.sims
counterfactual.impact.sims<-impact.factor.sims*counterfactual.mean.sims.matrix.after.correction*
rownames(current.impact.sims)<-impact.factor.names
rownames(counterfactual.impact.sims)<-impact.factor.names

##consumed
current.impact.sims.consumed<-impact.factor.sims*current.mean.sims.matrix*population.sims/as.num
counterfactual.impact.sims.consumed<-impact.factor.sims*counterfactual.mean.sims.matrix*populati
rownames(current.impact.sims.consumed)<-impact.factor.names
rownames(counterfactual.impact.sims.consumed)<-impact.factor.names

##unconsumed
current.impact.sims.unconsumed<-impact.factor.sims*current.mean.sims.matrix.unconsumed*populatio
counterfactual.impact.sims.unconsumed<-impact.factor.sims*counterfactual.mean.sims.matrix.uncons
rownames(current.impact.sims.unconsumed)<-impact.factor.names
rownames(counterfactual.impact.sims.unconsumed)<-impact.factor.names

# UNCOMMENT LATER
## inedible
current.impact.sims.inedible<-impact.factor.sims*current.mean.sims.matrix.inedible*population.si
counterfactual.impact.sims.inedible<-impact.factor.sims*counterfactual.mean.sims.matrix.inedible
rownames(current.impact.sims.inedible)<-impact.factor.names
rownames(counterfactual.impact.sims.inedible)<-impact.factor.names
## wasted
current.impact.sims.wasted<-impact.factor.sims*current.mean.sims.matrix.wasted*population.sims/a
counterfactual.impact.sims.wasted<-impact.factor.sims*counterfactual.mean.sims.matrix.wasted*pop
rownames(current.impact.sims.wasted)<-impact.factor.names
rownames(counterfactual.impact.sims.wasted)<-impact.factor.names

```

Calculating total deltas at population levels by multiplying our mean deltas with population counts

```

# population.sims.output<-as.matrix(population.sims)
# if(dim(population.sims.output)[1]>1)
# {
#   population.sims.output<-population.sims.output[rep(1, times=length(impact.factor.names)),] #
#   rownames(population.sims.output)<-impact.factor.names
# }
# if(dim(population.sims.output)[1]==1)
# {
#   rownames(population.sims.output)<-impact.factor.names
# }
#
#
# delta.sims.output<-as.matrix(delta.sims.matrix*population.sims)
# if(dim(delta.sims.output)[1]>1)
# {
#   delta.sims.output<-delta.sims.output[rep(1, times=length(impact.factor.names)),] ##repeat th
#   rownames(delta.sims.output)<-impact.factor.names
# }
# if(dim(delta.sims.output)[1]==1)
# {
#   rownames(delta.sims.output)<-impact.factor.names
# }

##Now population.sims.output and delta.sims.output are already matrices, don't need above code t
##Later, can decide if we want to keep pure deltas, or the modified deltas we use now

population.sims.output<-as.matrix(population.sims)
rownames(population.sims.output)<-impact.factor.names

#delta.sims.output<-as.matrix(delta.sims.matrix*population.sims)
#rownames(delta.sims.output)<-impact.factor.names

delta.sims.output.consumed<-as.matrix(delta.sims.matrix.consumed*population.sims)
rownames(delta.sims.output.consumed)<-impact.factor.names
delta.sims.output.unconsumed<-as.matrix(delta.sims.matrix.unconsumed*population.sims)
rownames(delta.sims.output.unconsumed)<-impact.factor.names
delta.sims.output.total<-as.matrix(delta.sims.matrix.total*population.sims)
rownames(delta.sims.output.total)<-impact.factor.names

# UNCOMMENT LATER
delta.sims.output.inedible<-as.matrix(delta.sims.matrix.inedible*population.sims)
rownames(delta.sims.output.inedible)<-impact.factor.names
delta.sims.output.wasted<-as.matrix(delta.sims.matrix.wasted*population.sims)
rownames(delta.sims.output.wasted)<-impact.factor.names

```

Take all output created and put it into a list of 31 matrices, each corresponding to simulations for a specific output of interest for the particular dietary factor - counterfactual diet pair, and return that list

```

# impact.list<-list("impact.sims" = impact.sims,
#                  "substitution.sims" = substitution.sims,
#                  "combined.impact.sims" = combined.impact.sims,
#                  "population.sims" = population.sims.output,
#                  "delta.sims" = delta.sims.output)

impact.list<-list("impact.sims.total" = impact.sims.total,
                  "substitution.sims.total" = substitution.sims.total,
                  "combined.impact.sims.total" = combined.impact.sims.total,
                  "population.sims.total" = population.sims.output,
                  "delta.sims.total" = delta.sims.output.total,

                  "impact.sims.consumed" = impact.sims.consumed,
                  "substitution.sims.consumed" = substitution.sims.consumed,
                  "combined.impact.sims.consumed" = combined.impact.sims.consumed,
                  "delta.sims.consumed" = delta.sims.output.consumed,

                  "impact.sims.unconsumed" = impact.sims.unconsumed,
                  "substitution.sims.unconsumed" = substitution.sims.unconsumed,
                  "combined.impact.sims.unconsumed" = combined.impact.sims.unconsumed,
                  "delta.sims.unconsumed" = delta.sims.output.unconsumed,

                  "current.intake.impact.total" = current.impact.sims,
                  "CF.intake.impact.total" = counterfactual.impact.sims,
                  "current.intake.impact.consumed" = current.impact.sims.consumed,
                  "CF.intake.impact.consumed" = counterfactual.impact.sims.consumed,
                  "current.intake.impact.unconsumed" = current.impact.sims.unconsumed,
                  "CF.intake.impact.unconsumed" = counterfactual.impact.sims.unconsumed,

                  #UNCOMMENT LATER,
                  "impact.sims.inedible" = impact.sims.inedible,
                  "substitution.sims.inedible" = substitution.sims.inedible,
                  "combined.impact.sims.inedible" = combined.impact.sims.inedible,
                  "delta.sims.inedible" = delta.sims.output.inedible,

                  "impact.sims.wasted" = impact.sims.wasted,
                  "substitution.sims.wasted" = substitution.sims.wasted,
                  "combined.impact.sims.wasted" = combined.impact.sims.wasted,
                  "delta.sims.wasted" = delta.sims.output.wasted,

                  "current.intake.impact.inedible" = current.impact.sims.inedible,
                  "CF.intake.impact.inedible" = counterfactual.impact.sims.inedible,
                  "current.intake.impact.wasted" = current.impact.sims.wasted,
                  "CF.intake.impact.wasted" = counterfactual.impact.sims.wasted
                  )

return(impact.list)
}

```

Next is a function that calculate per capita sims by dividing impact sims with population count.


```
get.percapita.sims<-function(impact.sims, population.sims, pop.sims.names, new.pop.sims.names, sim
{
  names(population.sims)[names(population.sims) %in% pop.sims.names]<-new.pop.sims.names ##replace
  impact.sims.plus.pop<-merge(impact.sims, population.sims)

  impact.sims.percapita<-impact.sims.plus.pop
  impact.sims.percapita[,sims.names]<-impact.sims.percapita[,sims.names]/impact.sims.percapita[,ne
  impact.sims.percapita<-impact.sims.percapita[,!(names(impact.sims.percapita) %in% new.pop.sims.n

  return(impact.sims.percapita)
}
```

Next, a function that calculates summary stats from simulation outputs, taking in simulation results for a specific diet factor as an input. Returns a list of 22 datatables, each corresponding to a different outcome (deltas, impact, substitution impact, combined impact, etc...)

```

#Let's make a functino out of this, since we're gonna repeat this whole thing for per capita
get.summary.stats<-function(impact.sims, substitution.impact.sims, combined.impact.sims, delta.sim
    impact.sims.consumed, substitution.impact.sims.consumed, combined.impa
    impact.sims.unconsumed, substitution.impact.sims.unconsumed, combined.
    current.intake.impact.sims, current.intake.impact.sims.consumed, curre
    CF.intake.impact.sims, CF.intake.impact.sims.consumed, CF.intake.impact
    impact.sims.inedible, substitution.impact.sims.inedible, combined.impa
    impact.sims.wasted, substitution.impact.sims.wasted, combined.impact.s
    current.intake.impact.sims.inedible, current.intake.impact.sims.wasted
    CF.intake.impact.sims.inedible, CF.intake.impact.sims.wasted,
    vars, convert.to.kcal.units="convert.to.kcal.units")
{
  impact.sims.dt<-as.data.table(impact.sims)
  impact.sims.summary <- impact.sims.dt[, "!="(mean_impact = rowMeans(.SD, na.rm = TRUE),
    sd_impact = apply(.SD, 1, sd),
    median_impact = apply(.SD, 1, median),
    lowerci_95_impact = apply(.SD, 1, function(x) quant
    upperci_95_impact = apply(.SD, 1, function(x) quant
    #lowerci_90_impact = apply(.SD, 1, function(x) quantile(x,
    #upperci_90_impact = apply(.SD, 1, function(x) quantile(x,
    .SDcols = vars
  ]
  impact.sims.summary <- impact.sims.summary[, (vars) := NULL]

  substitution.impact.sims.dt<-as.data.table(substitution.impact.sims)
  substitution.impact.sims.summary <- substitution.impact.sims.dt[, "!="(mean_substitution_impact
    sd_substitution_impact =
    median_substitution_impac
    lowerci_95_substitution_i
    upperci_95_substitution_i
    #lowerci_90_substitution_impact
    #upperci_90_substitution_impact
    .SDcols = vars
  ]
  substitution.impact.sims.summary <- substitution.impact.sims.summary[, (vars) := NULL]

  combined.impact.sims.dt<-as.data.table(combined.impact.sims)
  combined.impact.sims.summary <- combined.impact.sims.dt[, "!="(mean_combined_impact = rowMeans(.
    sd_combined_impact = apply(.SD, 1
    median_combined_impact = apply(.S
    lowerci_95_combined_impact = appl
    upperci_95_combined_impact = appl
    #lowerci_90_combined_impact = apply(.SD,
    #upperci_90_combined_impact = apply(.SD,
    .SDcols = vars
  ]
  combined.impact.sims.summary <- combined.impact.sims.summary[, (vars) := NULL]

  delta.sims.dt<-as.data.table(delta.sims)
  delta.sims.summary <- delta.sims.dt[, "!="(mean_delta = rowMeans(.SD, na.rm = TRUE),
    sd_delta = apply(.SD, 1, sd),
    median_delta = apply(.SD, 1, median),

```

```

        lowerci_95_delta = apply(.SD, 1, function(x) quantile
        upperci_95_delta = apply(.SD, 1, function(x) quantile
#lowerci_90_impact = apply(.SD, 1, function(x) quantile(x, .
#upperci_90_impact = apply(.SD, 1, function(x) quantile(x, .
.SDcols = vars
]
delta.sims.summary <- delta.sims.summary[, (vars) := NULL]

#####_consumed
impact.sims.consumed.dt<-as.data.table(impact.sims.consumed)
impact.sims.consumed.summary <- impact.sims.consumed.dt[, "!="(mean_impact_consumed = rowMeans(.
        sd_impact_consumed = apply(.SD, 1, sd),
        median_impact_consumed = apply(.SD, 1, median),
        lowerci_95_impact_consumed = apply(.SD, 1, function
        upperci_95_impact_consumed = apply(.SD, 1, function
#lowerci_90_impact = apply(.SD, 1, function(x) quantile(x,
#upperci_90_impact = apply(.SD, 1, function(x) quantile(x,
.SDcols = vars
]
impact.sims.consumed.summary <- impact.sims.consumed.summary[, (vars) := NULL]

substitution.impact.sims.consumed.dt<-as.data.table(substitution.impact.sims.consumed)
substitution.impact.sims.consumed.summary <- substitution.impact.sims.consumed.dt[, "!="(mean_su
        sd_substitution_impact_co
        median_substitution_impac
        lowerci_95_substitution_i
        upperci_95_substitution_i
#lowerci_90_substitution_impact
#upperci_90_substitution_impact
.SDcols = vars
]
substitution.impact.sims.consumed.summary <- substitution.impact.sims.consumed.summary[, (vars)

combined.impact.sims.consumed.dt<-as.data.table(combined.impact.sims.consumed)
combined.impact.sims.consumed.summary <- combined.impact.sims.consumed.dt[, "!="(mean_combined_i
        sd_combined_impact_consumed = app
        median_combined_impact_consumed =
        lowerci_95_combined_impact_consum
        upperci_95_combined_impact_consum
#lowerci_90_combined_impact = apply(.SD,
#upperci_90_combined_impact = apply(.SD,
.SDcols = vars
]
combined.impact.sims.consumed.summary <- combined.impact.sims.consumed.summary[, (vars) := NULL]

#####unconsumed
impact.sims.unconsumed.dt<-as.data.table(impact.sims.unconsumed)
impact.sims.unconsumed.summary <- impact.sims.unconsumed.dt[, "!="(mean_impact_unconsumed = rowM
        sd_impact_unconsumed = apply(.SD,
        median_impact_unconsumed = apply(
        lowerci_95_impact_unconsumed = ap
        upperci_95_impact_unconsumed = ap

```

```

#lowerci_90_impact = apply(.SD, 1, funct
#upperci_90_impact = apply(.SD, 1, funct
.SDcols = vars

]
impact.sims.unconsumed.summary <- impact.sims.unconsumed.summary[, (vars) := NULL]

substitution.impact.sims.unconsumed.dt<-as.data.table(substitution.impact.sims.unconsumed)
substitution.impact.sims.unconsumed.summary <- substitution.impact.sims.unconsumed.dt[, "!="(mean_
sd_subs
median_
lowerci
upperci
#lowerci_90_su
#upperci_90_su
.SDcols = vars

]
substitution.impact.sims.unconsumed.summary <- substitution.impact.sims.unconsumed.summary[, (va

combined.impact.sims.unconsumed.dt<-as.data.table(combined.impact.sims.unconsumed)
combined.impact.sims.unconsumed.summary <- combined.impact.sims.unconsumed.dt[, "!="(mean_combin
sd_combined_imp
median_combined
lowerci_95_comb
upperci_95_comb
#lowerci_90_combined_i
#upperci_90_combined_i
.SDcols = vars

]
combined.impact.sims.unconsumed.summary <- combined.impact.sims.unconsumed.summary[, (vars) := N

##### current.intake

current.intake.impact.sims.dt<-as.data.table(current.intake.impact.sims)
current.intake.impact.sims.summary <- current.intake.impact.sims.dt[, "!="(mean_impact_current_i
sd_impact_current_intake
median_impact_current_int
lowerci_95_impact_current
upperci_95_impact_current
#lowerci_90_substitution_impact
#upperci_90_substitution_impact
.SDcols = vars

]
current.intake.impact.sims.summary <- current.intake.impact.sims.summary[, (vars) := NULL]

current.intake.impact.consumed.sims.dt<-as.data.table(current.intake.impact.sims.consumed)
current.intake.impact.consumed.sims.summary <- current.intake.impact.consumed.sims.dt[, "!="(mea
sd_
med
low
upp
#lowerci_9
#upperci_9

```

```

                                                                    .SDcols =
]
current.intake.impact.consumed.sims.summary <- current.intake.impact.consumed.sims.summary[, (vars) := NULL]

current.intake.impact.unconsumed.sims.dt<-as.data.table(current.intake.impact.sims.unconsumed)
current.intake.impact.unconsumed.sims.summary <- current.intake.impact.unconsumed.sims.dt[, ":(vars) := NULL"
                                                                    sd_
                                                                    med
                                                                    low
                                                                    upp
                                                                    #Lowerci_9
                                                                    #upperci_9
                                                                    .SDcols =
]
current.intake.impact.unconsumed.sims.summary <- current.intake.impact.unconsumed.sims.summary[, (vars) := NULL]

#####CF.intake. _CF_intake
CF.intake.impact.sims.dt<-as.data.table(CF.intake.impact.sims)
CF.intake.impact.sims.summary <- CF.intake.impact.sims.dt[, ":(vars) := NULL"
                                                                    sd_impact_CF_intake = apply(.SD, 1, sd),
                                                                    median_impact_CF_intake = apply(.SD, 1, median),
                                                                    lowerci_95_impact_CF_intake = apply(.SD, 1, function(x) quantile(x, 0.05)),
                                                                    upperci_95_impact_CF_intake = apply(.SD, 1, function(x) quantile(x, 0.95)),
                                                                    #Lowerci_90_impact = apply(.SD, 1, function(x) quantile(x, 0.1)),
                                                                    #upperci_90_impact = apply(.SD, 1, function(x) quantile(x, 0.9)),
                                                                    .SDcols = vars
]
CF.intake.impact.sims.summary <- CF.intake.impact.sims.summary[, (vars) := NULL]

CF.intake.impact.consumed.sims.dt<-as.data.table(CF.intake.impact.sims.consumed)
CF.intake.impact.consumed.sims.summary <- CF.intake.impact.consumed.sims.dt[, ":(vars) := NULL"
                                                                    sd_
                                                                    med
                                                                    low
                                                                    upp
                                                                    #Lowerci_9
                                                                    #upperci_9
                                                                    .SDcols =
]
CF.intake.impact.consumed.sims.summary <- CF.intake.impact.consumed.sims.summary[, (vars) := NULL]

CF.intake.impact.unconsumed.sims.dt<-as.data.table(CF.intake.impact.sims.unconsumed)
CF.intake.impact.unconsumed.sims.summary <- CF.intake.impact.unconsumed.sims.dt[, ":(vars) := NULL"
                                                                    sd_
                                                                    med
                                                                    low
                                                                    upp
                                                                    #Lowerci_9
                                                                    #upperci_9
                                                                    .SDcols =
]
CF.intake.impact.unconsumed.sims.summary <- CF.intake.impact.unconsumed.sims.summary[, (vars) := NULL]

                                                                    #Lower
                                                                    #upper
                                                                    .SDcol

```

```

]
CF.intake.impact.unconsumed.sims.summary <- CF.intake.impact.unconsumed.sims.summary[, (vars) :=

####

#####inedible
impact.sims.inedible.dt<-as.data.table(impact.sims.inedible)
impact.sims.inedible.summary <- impact.sims.inedible.dt[, "!="(mean_impact_inedible = rowMeans(.
                                sd_impact_inedible = apply(.SD, 1, f
                                median_impact_inedible = appl
                                lowerci_95_impact_inedible =
                                upperci_95_impact_inedible =
                                #lowerci_90_impact = apply(.SD, 1, f
                                #upperci_90_impact = apply(.SD, 1, f
                                .SDcols = vars

]
impact.sims.inedible.summary <- impact.sims.inedible.summary[, (vars) := NULL]

substitution.impact.sims.inedible.dt<-as.data.table(substitution.impact.sims.inedible)
substitution.impact.sims.inedible.summary <- substitution.impact.sims.inedible.dt[, "!="(mean_su
                                sd_
                                med
                                low
                                upp
                                #Lowerci_9
                                #upperci_9
                                .SDcols =

]
substitution.impact.sims.inedible.summary <- substitution.impact.sims.inedible.summary[, (vars)

combined.impact.sims.inedible.dt<-as.data.table(combined.impact.sims.inedible)
combined.impact.sims.inedible.summary <- combined.impact.sims.inedible.dt[, "!="(mean_combined_i
                                sd_combined_impact_
                                median_combined_imp
                                lowerci_95_combined
                                upperci_95_combined
                                #Lowerci_90_combined_impac
                                #upperci_90_combined_impac
                                .SDcols = vars

]
combined.impact.sims.inedible.summary <- combined.impact.sims.inedible.summary[, (vars) := NULL]

#####wasted
impact.sims.wasted.dt<-as.data.table(impact.sims.wasted)
impact.sims.wasted.summary <- impact.sims.wasted.dt[, "!="(mean_impact_wasted = rowMeans(.SD, na
                                sd_impact_wasted = apply(.SD, 1,
                                median_impact_wasted = apply(.SD,
                                lowerci_95_impact_wasted = apply(
                                upperci_95_impact_wasted = apply(
                                #Lowerci_90_impact = apply(.SD, 1, funct
                                #upperci_90_impact = apply(.SD, 1, funct

```

```

        .SDcols = vars
    ]
    impact.sims.wasted.summary <- impact.sims.wasted.summary[, (vars) := NULL]

    substitution.impact.sims.wasted.dt<-as.data.table(substitution.impact.sims.wasted)
    substitution.impact.sims.wasted.summary <- substitution.impact.sims.wasted.dt[, "!="(mean_substi
                                                    sd_subs
                                                    median_
                                                    lowerci
                                                    upperci
                                                    #lowerci_90_su
                                                    #upperci_90_su
                                                    .SDcols = vars
    ]
    substitution.impact.sims.wasted.summary <- substitution.impact.sims.wasted.summary[, (vars) := N

    combined.impact.sims.wasted.dt<-as.data.table(combined.impact.sims.wasted)
    combined.impact.sims.wasted.summary <- combined.impact.sims.wasted.dt[, "!="(mean_combined_impac
                                                    sd_combined
                                                    median_comb
                                                    lowerci_95_
                                                    upperci_95_
                                                    #lowerci_90_combin
                                                    #upperci_90_combin
                                                    .SDcols = vars
    ]
    combined.impact.sims.wasted.summary <- combined.impact.sims.wasted.summary[, (vars) := NULL]

    all.impact.sims.summary<-Reduce(function(...) merge(..., all = T),list(delta.sims.summary,
                                                    impact.sims.summary, subs
                                                    impact.sims.consumed.summ
                                                    impact.sims.unconsumed.su
                                                    current.intake.impact.sim
                                                    CF.intake.impact.sims.sum
                                                    impact.sims.inedible.summ
                                                    impact.sims.wasted.summar

    return(all.impact.sims.summary)
}

```

Simple version of above function to use on just one set of outputs (as opposed to 22)

```

get.summary.stats.simple<-function(impact.sims, vars)
{
  impact.sims.dt<-as.data.table(impact.sims)
  impact.sims.summary <- impact.sims.dt[, "!="(mean_impact = rowMeans(.SD, na.rm = TRUE),
                                             sd_impact = apply(.SD, 1, sd),
                                             median_impact = apply(.SD, 1, median),
                                             lowerci_95_impact = apply(.SD, 1, function(x) quant
                                             upperci_95_impact = apply(.SD, 1, function(x) quant
#lowerci_90_impact = apply(.SD, 1, function(x) quantile(x,
#upperci_90_impact = apply(.SD, 1, function(x) quantile(x,
.SDcols = vars

]
  impact.sims.summary <- impact.sims.summary[, (vars) := NULL]

  return(impact.sims.summary)
}

```

And finally, a function that sums the output by all strata combinations of interest and outputs summary stats. Note that input "strata.combos.list" is a list of the strata combinations of interest, with each element being a string vector denoting the strata for that combination.

```

summary.stats.by.strata<-function(impact.sims.allgroups, strata.combos.list, pop.sims)
{

```

Convert input sims to data tables, create some string vectors for labeling simulated values, and create lists for storing results.

```

cols.to.sum<-paste("X", 1:n.sims, sep="")
pop.cols.to.sum<-paste("pop", 1:n.sims, sep="")

sims.data.table<-as.data.table(impact.sims.allgroups)
pop.sims.data.table<-as.data.table(pop.sims)

strata.combos.sims<-list()
strata.combos.sims.long<-list()
strata.combos.summary.stats.long<-list()
strata.combos.summary.stats.wide<-list()
strata.combos.summary.stats.misc<-list()

pop.strata.combos.sims<-list()

percapita.sims<-list()
percapita.sims.long<-list()
percapita.summary.stats.long<-list()
percapita.summary.stats.wide<-list()
percapita.summary.stats.misc<-list()

```

Start a for loop to traverse through each strata combination of interest.


```
for(j in 1:length(strata.combos.list))
{
```

Sum by combinations of strata in strata.combos.list[[j]]

```
labels<-c(strata.combos[[j]], "outcome", "outcome_unit")

strata.combos.sims[[j]]<-sims.data.table[,lapply(.SD, sum), by=c(strata.combos.list[[j]], "out
.SDcols=cols.to.sum]
```

Sum population sims by combinations of strata in strata.combos.list[[j]]. Note that we can't sum population counts by "Foodgroup" or "broad_foodgroup" since we would be double (or triple, quadruple) counting (these are not population attributes) if we did that, so we remove those from the strata combination when summing population counts. If the strata combination of interest is just the foodgroups, then we sum pop counts are summed over all strata.

```
pop.strata<-strata.combos[[j]][!(strata.combos.list[[j]] %in% c("Foodgroup", "broad_foodgroup")

if(is.null(pop.strata))
{
  pop.strata.combos.sims[[j]]<-pop.sims.data.table[,lapply(.SD, sum), .SDcols=cols.to.sum]
}else
{
  pop.strata.combos.sims[[j]]<-pop.sims.data.table[,lapply(.SD, sum), by=c(pop.strata), .SDcol
}
names(pop.strata.combos.sims[[j]])<-gsub(x=names(pop.strata.combos.sims[[j]]), pattern="X", re
```

merge the summed inputs and pop counts into one file.

```
if(length(intersect(names(strata.combos.sims[[j]]), names(pop.strata.combos.sims[[j]])))==0)
{
  merged<-cbind(strata.combos.sims[[j]], pop.strata.combos.sims[[j]])
}
if(length(intersect(names(strata.combos.sims[[j]]), names(pop.strata.combos.sims[[j]])))!=0)
{
  merged<-merge(strata.combos.sims[[j]], pop.strata.combos.sims[[j]])
}
```

Calculate per capita results of sims summed by strata

```
merged.names<-merged[,..labels]
percapita.sims[[j]]<-merged[,..cols.to.sum]/merged[,..pop.cols.to.sum]
percapita.sims[[j]]<-cbind(merged.names, percapita.sims[[j]])
```

Reshape sims into long format, calculate summary stats, and label outcomes appropriately, then reshape back to wide format so that each summary stat has its own column.

```

strata.combos.sims.long[[j]]<-melt(strata.combos.sims[[j]], id.vars = c(strata.combos[[j]], "out
strata.combos.summary.stats.long[[j]]<-strata.combos.sims.long[[j]][, .(quantile(value, c(.025
strata.combos.summary.stats.misc[[j]]<-strata.combos.sims.long[[j]][, .(mean(value), sd(value)
names(strata.combos.summary.stats.misc[[j]])[(dim(strata.combos.summary.stats.misc[[j]))[2]-1)

strata.combos.summary.stats.long[[j]]<-cbind(strata.combos.summary.stats.long[[j]],
      as.factor(rep(c("lower_bound (2.5th percentile)",
#strata.combos.summary.stats.wide[[j]]<-dcast(strata.combos.summary.stats.long[[j]], "Comorbid
strata.combos.summary.stats.wide[[j]]<-dcast(strata.combos.summary.stats.long[[j]], paste(paste
      value.var="V1")
#strata.combos.summary.stats.wide[[j]]<-merge(strata.combos.summary.stats.wide[[j]], strata.co
strata.combos.summary.stats.wide[[j]]<-merge(strata.combos.summary.stats.wide[[j]], strata.com
      by=intersect(names(strata.combos.summary.stats.wide[[j]]

```

Repeat for per capita sims.

```

percapita.sims.long[[j]]<-melt(percapita.sims[[j]], id.vars = c(strata.combos.list[[j]], "outc
percapita.summary.stats.long[[j]]<-percapita.sims.long[[j]][, .(quantile(value, c(.025, .5, .9
percapita.summary.stats.misc[[j]]<-percapita.sims.long[[j]][, .(mean(value), sd(value)), by=c(
names(percapita.summary.stats.misc[[j]])[(dim(strata.combos.summary.stats.misc[[j]))[2]-1):dim

percapita.summary.stats.long[[j]]<-cbind(percapita.summary.stats.long[[j]],
      as.factor(rep(c("lower_bound (2.5th percentile)", "me
      dim(percapita.summary.stats.long[[j]])[
#strata.combos.summary.stats.wide[[j]]<-dcast(strata.combos.summary.stats.long[[j]], "Comorbid
percapita.summary.stats.wide[[j]]<-dcast(percapita.summary.stats.long[[j]], paste(paste(strata
      value.var="V1")
percapita.summary.stats.wide[[j]]<-merge(percapita.summary.stats.wide[[j]], percapita.summary.
      by=intersect(names(percapita.summary.stats.wide[[j]])

```

End loop

```

}
```

After looping through all strata combos of interest, we return the result as a 4 element list. 1 = summary output, 2 = full sims, 3 = per capita summary output, 4 = per capita full sims. Each of these 4 elements are themselves lists with with element corresponding to a strata combination of interest.

```

return(list("summary.output" = strata.combos.summary.stats.wide,
      "full.sims.output" = strata.combos.sims,
      "summary.output.percapita" = percapita.summary.stats.wide,
      "full.sims.output.percapita" = percapita.sims)
)
}
```